

Advanced AI Medical Intelligence Platform

Project Report

Deep Learning • Explainable AI • LLM-Assisted Reporting • REST API •
Deployment

Prepared as a technical portfolio project.

1. Project Objective

This project implements a complete, end-to-end AI application for medical image analysis. It classifies chest X-ray images as NORMAL or PNEUMONIA using a deep convolutional neural network, explains each prediction visually using Grad-CAM (Gradient-weighted Class Activation Mapping), generates a plain-language draft report using a Large Language Model (Google Gemini), exposes all functionality through a documented REST API, persists every prediction in a relational database, and ships with a Streamlit-based web interface plus Docker-based deployment.

Disclaimer: This system is an educational/portfolio project. It is not a certified medical device and is not intended for real clinical diagnosis. Every AI-generated report explicitly states this limitation.

2. System Architecture

The system follows a modular, three-tier architecture:

- **Model layer** (model/): ResNet18 CNN, training pipeline, Grad-CAM explainability module, dataset preparation utilities.
- **Backend layer** (backend/): FastAPI REST API, SQLAlchemy ORM for database persistence, Gemini-based LLM report generator, inference pipeline tying the model and Grad-CAM together.
- **Frontend layer** (frontend/): Streamlit web application for image upload, result visualization, and history browsing.

Both backend and frontend are independently containerized and orchestrated together via Docker Compose, so the entire platform can be brought up with a single command.

3. Dataset

The model is designed for the public 'Chest X-Ray Images (Pneumonia)' dataset (Kermany et al.), containing labeled NORMAL and PNEUMONIA chest radiographs split into train/validation/test sets. A synthetic data generator (model/dataset_prep.py --synthetic) is also included, producing procedurally generated X-ray-like images so the entire pipeline (training, inference, Grad-CAM, API, database, UI) can be verified end-to-end without requiring the ~2GB download, which is useful for CI/demo purposes.

4. Deep Learning Model

Component	Detail
Architecture	ResNet18 (ImageNet-pretrained backbone, fine-tuned FC head)
Framework	PyTorch / TorchVision
Input	224 x 224 RGB image

Output classes	NORMAL, PNEUMONIA (binary; extensible to multi-class)
Loss function	Cross-Entropy Loss
Optimizer	Adam (lr=1e-4) with StepLR scheduler
Augmentation	Random horizontal flip, random rotation (+/-10 deg)
Evaluation metrics	Accuracy, Precision, Recall, F1-score, Confusion Matrix

Training and test metrics are automatically written to saved_models/training_history.json and saved_models/metrics.json after each run, so performance is fully reproducible and auditable.

4.1 Training Results on the Real Kaggle Dataset

The model was trained end-to-end on the real 'Chest X-Ray Images (Pneumonia)' dataset (5,216 training images, 624 held-out test images: 234 NORMAL / 390 PNEUMONIA) for 10 epochs using the pipeline described above. Final test-set performance is summarized below.

Metric	Value	Interpretation
Accuracy	85.4%	533 of 624 test images classified correctly
Precision	81.1%	Of all PNEUMONIA predictions, 81% were correct
Recall (Sensitivity)	100.0%	Every true PNEUMONIA case in the test set was caught (zero false negatives)
F1-score	89.6%	Harmonic mean of precision and recall

Confusion Matrix (rows = actual, columns = predicted):

	Predicted: NORMAL	Predicted: PNEUMONIA
Actual: NORMAL	143	91
Actual: PNEUMONIA	0	390

Clinical interpretation: A recall of 100% means the model produced zero false negatives on this test set — every genuine pneumonia case was flagged. This is the property most valued in a medical screening context, since missing a true positive (a sick patient marked healthy) is far costlier than a false alarm. The trade-off is a lower precision (81.1%): 91 healthy (NORMAL) X-rays were incorrectly flagged as PNEUMONIA, which would trigger unnecessary follow-up but not a missed diagnosis. This conservative bias is consistent with the training set's class imbalance (3,875 PNEUMONIA vs 1,341 NORMAL images) and is in line with accuracy ranges (85-93%) reported in published work using this same public dataset.

5. Explainable AI (Grad-CAM)

To avoid a 'black box' prediction, every inference request also runs Grad-CAM on the final convolutional block (layer4) of the ResNet18 backbone. Grad-CAM computes gradients of the

predicted class score with respect to the feature maps of that layer, weights the activations accordingly, and produces a heatmap highlighting the image regions most responsible for the prediction. This heatmap is overlaid on the original X-ray and returned alongside every prediction — giving a visual, clinically-interpretable explanation rather than a bare probability score.

6. LLM-Assisted Report Generation

The backend integrates Google's Gemini API (via the google-genai SDK) to translate the raw model output (predicted class + confidence) into a short, structured, plain-language draft report with four sections: Summary, Observations, Recommended Next Steps, and a mandatory Disclaimer clarifying that the output is an AI-assisted screening aid, not a diagnosis. If no API key is configured, the system gracefully falls back to a templated disclaimer-only report rather than failing, so the application remains usable end-to-end in any environment.

7. REST API Design

Method	Endpoint	Purpose
GET	/health	Health check
POST	/predict	Upload image -> prediction + Grad-CAM + LLM report
GET	/history	Paginated list of past predictions
GET	/history/{id}	Full detail of one prediction
DELETE	/history/{id}	Delete a prediction record
GET	/gradcam/{filename}	Serve a saved Grad-CAM heatmap image

Built with FastAPI, the API is fully documented via auto-generated OpenAPI/Swagger docs at /docs, uses Pydantic for request/response validation, and includes CORS middleware for frontend integration.

8. Database Design

Prediction history is persisted using SQLAlchemy ORM against a SQLite database by default (zero-config), with a single DATABASE_URL environment variable swap needed to move to PostgreSQL or MySQL in production. Each record stores: id, filename, predicted class, confidence score, path to the saved Grad-CAM heatmap image, the generated LLM report text, and a UTC timestamp.

9. Web Application (Streamlit)

The frontend provides two views: (1) a New Prediction tab for uploading an X-ray, triggering analysis, and viewing the prediction, confidence, Grad-CAM overlay, and LLM report side-by-side; and (2) a

History tab listing all past predictions with expandable detail (heatmap + report) and delete functionality, fetched live from the REST API.

10. Deployment

The project includes a Dockerfile for the FastAPI backend and a separate Dockerfile.frontend for the Streamlit app, orchestrated together with docker-compose.yml. A single 'docker compose up --build' command builds and starts both services, with the trained model checkpoint, Grad-CAM output directory, and SQLite database file mounted as persistent volumes.

11. Testing

A pytest-based smoke test suite (tests/test_api.py) uses FastAPI's TestClient to verify the health check, the full predict -> history -> detail -> delete workflow, and input validation (rejecting non-image uploads). This suite was executed during development and passes end-to-end, confirming the model, Grad-CAM, database, and LLM fallback path all integrate correctly.

12. Best Practices Followed

- Modular, separation-of-concerns project structure (model / backend / frontend)
- Environment-variable-driven configuration (.env.example) — no hardcoded secrets
- Graceful degradation: app remains functional without a Gemini API key or GPU
- Input validation and structured error handling on all API endpoints
- Automated tests and reproducible training metrics
- Containerized, one-command deployment via Docker Compose

13. Future Work

- Multi-class disease classification (COVID-19, tuberculosis, etc.)
- User authentication (JWT) for multi-user prediction history
- PostgreSQL + Alembic migrations for production-scale deployment
- CI/CD pipeline (GitHub Actions) running the test suite on every push
- Model experiment tracking with MLflow or Weights & Biases